HITBOT

hitbot@hitbot.cc
www.hitbot.cc

Z-Arm-API-WINDOWS-C++

领先的轻量型协作机械臂提供商

主营: 工业机器人 /协作机器人 /电动夹爪 /电缸模组

慧灵科技（深圳）有限公司
Huiling-tech Robotic Co., LTD

发布说明

日期	版本	发布说明
2018.07.03	1.0.0.20180703_release	首次发布
2018.08.20	1.0.0.20180819_beta	添加五轴相关函数
2018.08.31	1.0.0.20180831_beta	修改 net_port_initial 返回值说明
2020.2.28	2.0.0.20200208_beta	1-添加第五轴相关接口 2-常规更新
2020.7.15	2.0.0.20200715_beta	1-添加 new_movej_xyz_lr 运动接口 2-添加 set_new_acc 接口
2022.3.8	2.0.0.20220308_beta	1. 新增电动夹爪控制接口； 2. 急停状态获取，清除；
2022.6.20	2.0.0.20220620_beta	1. 新增新版末端 IO 板输出类型设置接口；
2022.7.09	2.0.0.20220709_beta	1. 新增控制抱闸开合和获取抱闸状态设置接口；
2023.10.14	2.0.0.20231014_beta	1. 新增函数调用示例;
2024.6.18	2.0.0.20240618_beta	1. 增加 485 夹爪功能设置函数;

目录

1、准备工作 6

2、初始化流程 6

3、SDK 接口 7

3.1 hitbot_interface.h： 7

3.1.1 导出函数： 8

3.1.1.1 net_port_initial 8
3.1.1.2 card_number_connect 8
3.1.1.3 get_robot 9
3.1.1.4 close_tcpserver 9

3.2 ControlBeanEx 类： 10

3.2.1 ControlBeanEx 成员变量： 10
3.2.1.1 机械臂当前状态： 10
3.2.1.2 位置预判 11
3.2.2 ControlBeanEx 成员函数： 12
3.2.2.1 initial 12
3.2.2.2 get_scara_param 14
3.2.2.3 ~~set_arm_length~~(过时，不推荐使用) 14
3.2.2.4 int unlock_position 15
3.2.2.5 is_connected 16
3.2.2.6 get_joint_state 17
3.2.2.7 set_drag_teach 19
3.2.2.8 get_drag_teach 19
3.2.2.9 set_cooperation_fun_state 20
3.2.2.10 get_cooperation_fun_state 21
3.2.2.11 is_collision 21
3.2.2.12 ~~set_position_move~~(过时，推荐使用 new_movej_xyz_lr) 22
3.2.2.13 ~~set_angle_move~~(过时，推荐使用 new_movej_xyz_lr) 24
3.2.2.14 ~~xyz_move~~(过时，推荐使用 new_movej_xyz_lr) 26
3.2.2.15 ~~single_joint_move~~(过时，推荐使用 new_movej_xyz_lr) 27
3.2.2.16 trail_move 29
3.2.2.17 ~~stop_move~~(过时，不推荐使用) 31

3.2.2.18 is_robot_goto_target 31
3.2.2.19 set_allow_offset_at_target_position 32
3.2.2.20 set_catch_or_release_accuracy 33
3.2.2.21 judge_in_range 33
3.2.2.22 ~~judge_position_gesture~~(过时，不推荐使用) 34
3.2.2.23 joint_home 35
3.2.2.24 movel_xyz 36
3.2.2.25 ~~movej_xyz~~(过时，推荐使用 new_movej_xyz_lr) 38
3.2.2.26 ~~movej_xyz_lr~~(过时，推荐使用 new_movej_xyz_lr) 40
3.2.2.27 ~~movej_angle~~(过时，推荐使用 new_movej_xyz_lr) 42
3.2.2.28 change_attitude 44
3.2.2.29 wait_stop 45
3.2.2.30 pause_move 46
3.2.2.31 resume_move 46
3.2.2.32 set_digital_out 47
3.2.2.33 get_digital_out 48
3.2.2.34 get_digital_in 48
3.2.2.35 set_efg_state 49
3.2.2.36 get_efg_state 50
3.2.2.37 new_movej_xyz_lr 51
3.2.2.38 new_set_acc 53
3.2.2.39 j5_motor_zero 54
3.2.2.40 set_j5_motor_pos 55
3.2.2.41 get_j5_parameter 56
3.2.2.42 movej_j5 57
3.2.2.43 ~~get_j5_positon~~（过时，建议使用 get_j5_paramter) 58
3.2.2.44 get_efg_state_dji 58
3.2.2.45 set_efg_state_dji 60
3.2.2.46 new_stop_move 61
3.2.2.47 new_movej_angle 62
3.2.2.48 new_set_efg_state(新增双通道) 64
3.2.2.49 new_get_efg_state(新增双通道) 65
3.2.2.50 get_hard_emergency_stop_state 66
3.2.2.51 clear_hard_emergency_stop 67
3.2.2.52 set_io_card_out_type 68
3.2.2.53 set_brake_state 69
3.2.2.54 get_brake_state 70

3.2.2.55 set_tool_fun1 70
3.2.2.56 set_tool_fun2 71
3.2.2.57 set_tool_fun3 72
3.2.2.58 com485_initial 73
3.2.2.59 com485_send 74
3.2.2.60 com485_recv 75
3.2.2.61 set_user 76
3.1.2.62 com485_init 77
3.1.2.63 com485_set_rotation_speed 78
3.1.2.64 com485_set_clamping_speed 79
3.1.2.65 com485_set_clamping_distance 80
3.1.2.66 com485_get_clamping_distance 81
3.1.2.67 com485_set_rotation_angle 81
3.1.2.68 com485_set_relative_rotation_angle 82
3.1.2.69 com485_get_rotation_angle 83
3.1.2.70 com485_get_clamping_state 83
3.1.2.71 com485_get_rotation_state 84
3.1.2.72 com485_set_clamping_current 85
3.1.2.73 com485_set_rotation_current 86
3.1.2.74 com485_get_clamping_current 87
3.1.2.75 com485_get_rotation_current 87
3.1.2.76 com485_suction_cup_control 87
3.1.2.77 com485_min_vacuum_degree 88
3.1.2.78 com485_max_vacuum_degree 89
3.1.2.79 com485_get_draw_state 90
3.1.2.80 get_error_code 91

目前可支持 windows 平台 CPP 语言开发的 x86 或者 x64 应用程序，可以使用 vs 或者 qt 进行开发。

1、准备工作

- 新建 cpp 工程，将 hitbot_interface.h，ControlBeanEx.h，hitbot_interface.lib 三个文件按复制到程序主程序源码目录下，添加前两个头文件到工程中，主程序中 include hitbot_interface.h 头文件；
- 将对应版本的库文件复制到 debug 文件夹下，包括 hitbot_interface.dll，share.dll，server.exe，small_scara_interface.dll 四个文件；

2、初始化流程

```
#include "hitbot_interface.h"
#include <iostream>

using namespace std;

#define ZARM_2140 1
#define ZARM_1632 5

int robot_id = 84;
ControlBeanEx *robot;

int main()
{
    /*
    *vs调试时，需要先将输出文件下的server.exe双击运行起来，否则程序将卡在
net_port_initial
    */
    net_port_initial();
    robot = get_robot(robot_id);

    int i = 0;
    for (i = 0; i < 500; i++)
```

```
    {
        if (robot->is_connected())
        {
            break;
        }
        Sleep(10);
    }

    if (i == 500)
    {
        cout << "connect time out" << endl;
    }
    else
    {
        int ret = robot->initial(ZARM_2140, 210);
        if (ret == 1)
        {
            robot->unlock_position();
            cout << "initial successful" << endl;

        }
    }

    return 1;
}
```

3、SDK 接口

3.1 hitbot_interface.h：

头文件	hitbot_interface.h
导出函数功能	申请内存和初始化网络服务器，获得机械臂控制实例对象等。

3.1.1 导出函数:

3.1.1.1 net_port_initial

功能	申请内存和初始化网络服务器
注意	调用其他 api 之前，需要先调用该函数，否则将导致程序崩溃，函数返回后，建议延迟 3s
函数定义	int net_port_initial()
参数	无
返回	•1 成功 •0 失败，一般是 40000 端口号被占用

代码示例:

```
int ret=net_port_initial();
if(ret==1)
{
    cout << "网络初始化成功" << endl;
}
else
{
    cout << "网络初始化失败" << endl;
}
```

3.1.1.2 card_number_connect

功能	查询指定的机械臂是否在线
注意	传入参数范围应>=0,<=255
函数定义	int card_number_connect(int card_number)
参数	int card_number：机械臂 ip 地址的第四位
返回	• 0：未连接

	• 1：已连接 • 2：传入参数错误 • 101：传入参数 NOT A NUMBER

代码示例：

```
ControlBeanEx *robot;
for (int i = 0; i < 255; i++)
{
    if (card_number_connect(i)==1) {
        robot = get_robot(i);
        cout << "connected id = " << i << endl;
        break;
    }
}
```

3.1.1.3 get_robot

功能	获取机械臂实例对象
注意	传入参数范围应>=0,<=255
函数定义	ControlBeanEx * get_robot(int card_number)
参数	int card_number：机械臂 ip 地址的第四位
返回	ControlBeanEx 实例对象指针

代码示例：

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
```

3.1.1.4 close_tcpserver

功能	关闭网络服务器
注意	此操作将立刻关闭程序与机器人的连接，机器人将立刻停止运动

函数定义	static void close_tcpserver()
参数	无
返回	无

代码示例：

```
close_tcpserver();
```

3.2 ControlBeanEx 类：

类名	ControlBeanEx
功能	通过成员函数和成员变量控制机械臂和获取机械臂当前状态。

3.2.1 ControlBeanEx 成员变量：

3.2.1.1 机械臂当前状态：

调用 get_scara_param 更新	
变量定义	功能
float x	x 坐标（mm）
float y	y 坐标（mm）
float z	z 坐标（mm）
float rotation	轴 4 角度（deg）
float goal_angle1	轴 1 角度（deg）
float goal_angle2	轴 2 角度（deg）

bool communicate_success	• true：不在线 • false：在线
bool initial_finish	• true：已初始化 • false：未初始化
bool move_flag	• true：正在运动 • false：静止状态

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
float x, y, z, goal_angle1, goal_angle2, rotation;
bool communicate_success,initial_finish,move_flag;
robot->get_scara_param();
x = robot->x;
y = robot->y;
z = robot->z;
goal_angle1 = robot->goal_angle1;
goal_angle2 = robot->goal_angle2;
rotation = robot->rotation;
communicate_success=robot->communicate_success;
initial_finish=robot->initial_finish;
move_flag=robot->move_flag;
```

3.2.1.2 位置预判

调用 judge_position_gesture 返回 ture 更新	
变量定义	功能

bool isReach_after_judge	• true：judge_position_gesture 预判的坐标点位可以到达 • false：judge_position_gesture 预判的坐标点位不在机器人的工作范围内
float angle1_after_judge	机器人在预判点位的轴 1 角度坐标（deg）
float angle2_after_judge	机器人在预判点位的轴 2 角度坐标（deg）

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
float x=320,float y=0;
bool ret=robot->judge_position_gesture(x, y);
if(ret==true)
{
    cout << "预判的坐标点位可以到达" << endl;
}
else
{
    cout << "预判的坐标点位不在机器人的工作范围内" << endl;
}
```

3.2.2 ControlBeanEx 成员函数:

3.2.2.1 initial

功能	初始化机械臂的各项参数，机械臂进入工作状态
注意	• 机械臂在运动时调用该函数，会造成机械臂抖动 • 返回 3 时，可以使用 joint_home 使关节强制回零
函数定义	int initial(int generation, float z_travel)

参数	• int generation：400mm 臂展系列传入 1，320mm 臂展系列传入 5 • float z_travel：上下关节有效行程（mm）
返回	• 0：机械臂不在线 • 1：初始化成功 • 2：generation 参数错误 • 3：机械臂当前位置不在限定范围，可以使用 joint_home 使关节强制回零 • 12：z_travel 传参错误 • 101：传入参数 NOT A NUMBER • 105：存在某一个关节失效 • >= 10000，pid 自检异常

代码示例:

```
#define ZARM_2140 1
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
int ret = robot->initial(ZARM_2140, 210);
if (ret == 1)
{
    cout << "initial successful" << endl;
}
else
{
    cout << "initial fail" << endl;
}
```

3.2.2.2 get_scara_param

功能	更新 x，y，z 等成员变量
注意	机械臂不在线时，坐标参数无意义
函数定义	void get_scara_param()
参数	无
返回	无

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
float x, y, z, goal_angle1, goal_angle2, rotation;
bool communicate_success,initial_finish,move_flag;
robot->get_scara_param();
x = robot->x;
y = robot->y;
z = robot->z;
goal_angle1 = robot->goal_angle1;
goal_angle2 = robot->goal_angle2;
rotation = robot->rotation;
communicate_success=robot->communicate_success;
initial_finish=robot->initial_finish;
move_flag=robot->move_flag;
```

3.2.2.3 ~~set_arm_length~~(过时，不推荐使用)

功能	设置机械臂轴 1 和轴 2 长度
注意	initial 函数中会按照设定机型自动设置该参数，如果臂长没有

	变化，无需重复调用
函数定义	void set_arm_length(float l1, float l2)
参数	• float l1：轴 1 长度（mm） • float l2：轴 2 长度（mm）
返回	• 0：机械臂不在线 • 1：初始化成功 • 2：generation 参数错误 • 3：机械臂当前位置不在限定范围 • 11：手机端正在控制，本次初始化不成功 • 12：z_travel 传参错误 • 101：传入参数 NOT A NUMBER

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
float l1=220;float l2=200;
robot = get_robot(robot_id);
robot->set_arm_length(l1,l2);
```

3.2.2.4 int unlock_position

功能	解锁机械臂，使机械臂可以接受运动指令
注意	一般初始化成功后立刻调用
函数定义	int unlock_position()

参数	无
返回	• 0：机械臂不在线 • 1：成功

代码示例:

```
#define ZARM_2140 1
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
int ret = robot->initial(ZARM_2140, 210);
if (ret == 1)
{
    cout << "initial successful" << endl;
    ret=robot->unlock_position();
    if(ret==1)
    {
        cout << "解锁成功" << endl;
    }
    else
    {
        cout << "解锁失败" << endl;
    }
}
else
{
    cout << "initial fail" << endl;
}
```

3.2.2.5 is_connected

功能	查询机械臂是否在线
注意	无
函数定义	bool is_connected()
参数	无
返回	• false：机械臂不在线

	• true：机械臂在线

代码示例:

```
net_port_initial();
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
bool ret=robot->is_connected();
if (ret == true)
{
    cout << "机械臂在线" << endl;
}
else
{
    cout << "机械臂不在线" << endl;
}
```

3.2.2.6 get_joint_state

功能	获取机械臂关节状态
注意	无
函数定义	int get_joint_state(int joint_num)
参数	int joint_num：轴号 1-4
返回	• 0：轴发生复位，需要重新初始化 • 1：关节正常 • 2：传入参数超范围 • 3：未初始化 • 4：轴状态获取失败 • 5：发生碰撞; • 6：处于拖动模式 • 7：关节缺轴/失效

	• 8：编码器错误 • 9：电机堵转/过电流保护 • 10：过压保护 • 11：手臂收到 PC 发送的错误数据 • 12：手臂主控与驱动通讯异常 • 21：MOS ntc 开路 • 22：MOS ntc 短路 • 23：电机 ntc 开路 • 24：电机 ntc 短路 • 25：MOS ntc 温度过高 • 26：MOS ntc 温度过低 • 27：电机 ntc 温度过高 • 28：电机 ntc 温度过低

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
int ret=robot->get_joint_state(1);
if (ret == 1)
{
    cout << "关节正常" << endl;
}
else
{
    cout << "关节异常" << endl;
}
```

3.2.2.7 set_drag_teach

功能	设定拖动示教功能是否开启
注意	• 仅协作机型支持开启此功能 • 拖动示教功能开启后，轴 3 不支持拖动，需要通过调用运动函数控制轴 3，
函数定义	bool set_drag_teach(bool state)
参数	bool state：true 开启，false 关闭
返回	• false：失败 • true：成功

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
bool ret=robot->set_drag_teach(true);
if (ret == true)
{
    cout << "拖动示教开启成功" << endl;
}
else
{
    cout << "拖动示教开启失败" << endl;
}
```

3.2.2.8 get_drag_teach

功能	查询是否处于拖动示教状态
注意	无
函数定义	bool get_drag_teach()
参数	无

返回	• false：开启 • true：关闭

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
bool ret=robot->get_drag_teach();
if (ret == true)
{
    cout << "拖动示教关闭" << endl;
}
else
{
    cout << "拖动示教开启" << endl;
}
```

3.2.2.9 set_cooperation_fun_state

功能	设定协作功能是否开启，开启后机械臂遇到障碍物会停止运动，并上报此状态
注意	仅协作机型支持开启此功能
函数定义	bool set_cooperation_fun_state(bool state)
参数	bool state：true 开启，false 关闭
返回	• false：成功 • true：失败

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
bool ret=robot->set_cooperation_fun_state(true);
if (ret == true)
{
    cout << "设定协作功能开启失败" << endl;
}
```

```
else
{
    cout << "设定协作功能开启成功" << endl;
}
```

3.2.2.10 get_cooperation_fun_state

功能	查询机械臂是否开启协作功能
注意	
函数定义	bool get_cooperation_fun_state()
参数	无
返回	• false：开启 • true：关闭

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
bool ret=robot->get_cooperation_fun_state();
if (ret == true)
{
    cout << "设定协作功能关闭" << endl;
}
else
{
    cout << "设定协作功能开启" << endl;
}
```

3.2.2.11 is_collision

功能	查询机械臂是否发生碰撞
注意	无
函数定义	bool is_collision()
参数	无

返回	• false：未发生碰撞 • true：发生碰撞

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
bool ret=robot->is_collision();
if (ret == true)
{
    cout << "发生碰撞" << endl;
}
else
{
    cout << "未发生碰撞" << endl;
}
```

3.2.2.12 ~~set_position_move~~(过时，推荐使用 new_movej_xyz_lr)

功能	驱动机械臂到达设定的 xyzr 坐标点
注意	• move_flag 等于 false 时调用有效 • movej 模式下，机械臂优先以当前姿态到达目标点
函数定义	int set_position_move(float goal_x, float goal_y, float goal_z, float goal_r, float speed, float acceleration, int interpolation, int move_mode)
参数	• float goal_x：目标点的 x 坐标值（mm）

	• float goal_y：目标点的 y 坐标值（mm） • float goal_z：目标点的 z 坐标值（mm） • float goal_r：目标点的 r 坐标值（deg） • float speed：xyz 点移动距离小于等于 1mm，轴 4 转动角度大于 0.0.1deg 时，speed 为轴 4 旋转速度（deg/s），其他关节做协同运动，其他情况 speed 为 xyz 点线速度（mm/s），轴 4 做协同运动 • float acceleration：T 型插补时的加速度（mm/s^2） • int interpolation：1 S 型插补，2 T 型插补 • int move_mode：1 角度运动（MOVEJ）,2 直线运动（MOVEL）
返回	• 0：正在执行其他指令，本次指令无效 • 1：本次指令生效，机械臂开始运动 • 2：设置速度小于等于零 • 3：未初始化 • 4：MOVEJ 运动模式下，目标点无法到达 • 6：伺服未开启 • 7：MOVEL 运动方式下，存在中间过程点无法以机械臂当前姿态（手系）达到 • 8：加速度小于等于零 • 9：插补方式参数错误 • 10：移动方式错误

	• 11：手机端在控制 • 101：传入参数 NOT A NUMBER • 102：发生碰撞，本次指令无效 • 103：轴发生复位，需要重新初始化，本次指令无效

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
float goal_x=420;
float goal_y=0;
float goal_z=0;
float goal_r=0;
float speed=100;
float acceleration=300;
int interpolation=1;
int move_mode=1;
int
ret=robot->set_position_move(goal_x,goal_y,goal_z,goal_r,speed,acceleration,interpolati
on,move_mode);
if (ret == 1)
{
    cout << "本次指令生效，机械臂开始运动" << endl;
}
else
{
    cout << "机械臂指令无法执行" << endl;
}
```

3.2.2.13 ~~set_angle_move~~(过时，推荐使用 new_movej_xyz_lr)

功能	驱动机械臂到达设定的 angle1，angle2，z，r 坐标点
注意	move_flag 等于 false 时调用有效
函数定义	int set_angle_move(float goal_angle1, float goal_angle2,

	float goal_z, float goal_r, float speed)
参数	• float goal_angle1：目标点轴 1 角度坐标值（deg） • float goal_angle2：目标点轴 2 角度坐标值（deg） • float goal_z：目标点 z 坐标值（mm） • float goal_r：目标点 r 坐标值（deg） • float speed：xyz 点移动距离小于等于 1mm，轴 4 转动角度大于 0.0.1deg 时，speed 为轴 4 旋转速度（deg/s），其他关节做协同运动，其他情况 speed 为 xyz 点线速度（mm/s），轴 4 做协同运动
返回	• 0：正在执行其他指令，本次指令无效 • 1：本次指令生效，机械臂开始运动 • 2：设置速度小于等于零 • 3：未初始化 • 4：MOVEJ 运动模式下，目标点无法到达 • 6：伺服未开启 • 11：手机端在控制 • 101：传入参数 NOT A NUMBER • 102：发生碰撞，本次指令无效 • 103：轴发生复位，需要重新初始化，本次指令无效

代码示例:

```
int robot_id = 84;
```

```
ControlBeanEx *robot;
robot = get_robot(robot_id);
float goal_angle1=20;
float goal_angle2=40;
float goal_z=0;
float goal_r=80;
float speed=100;
int ret=robot->set_angle_move(goal_angle1,goal_angle2,goal_z,goal_r,speed);
if (ret == 1)
{
    cout << "本次指令生效，机械臂开始运动" << endl;
}
else
{
    cout << "机械臂指令无法执行" << endl;
}
```

3.2.2.14 ~~xyz_move~~(过时，推荐使用 new_movej_xyz_lr)

功能	驱动机械臂沿某一方向运动指定距离
注意	• move_flag 等于 false 时调用有效 • 运动轨迹为直线
函数定义	int xyz_move(int direction, float distance, float speed)
参数	• int direction：1 x 轴正方向，2 y 轴正方向，3 z 轴正方向 • float distance：移动距离（mm） • float speed ：移动速度（mm/s）
返回	• 0：正在执行其他指令，本次指令无效 • 1：本次指令生效，机械臂开始运动

	• 2：设置速度小于等于零 • 3：未初始化 • 4：过程点无法到达 • 5：方向参数错误 • 6：伺服未开启 • 7：该姿态无法完成直线运动 • 11：手机端在控制 • 101：传入参数 NOT A NUMBER • 102：发生碰撞，本次指令无效 • 103：轴发生复位，需要重新初始化，本次指令无效

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
int direction=1;
float distance=30;
float speed=100;
int ret=robot->xyz_move(direction,distance,speed);
if (ret == 1)
{
    cout << "本次指令生效，机械臂开始运动" << endl;
}
else
{
    cout << "机械臂指令无法执行" << endl;
}
```

3.2.2.15 ~~single_joint_move~~(过时，推荐使用 new_movej_xyz_lr)

功能	驱动机械臂某一个轴旋转指定角度
注意	• move_flag 等于 false 时调用有效

函数定义	int single_joint_move(int axis, float distance, float speed)
参数	• int axis：1 轴 1 正方向，2 轴 2 正方向，3 轴 3 正方向，4 轴 4 正方向 • float distance：旋转角度（deg） • float speed ：轴 3 为线速度（mm/s），其他为旋转速度（deg/s）
返回	• 0：正在执行其他指令，本次指令无效 • 1：本次指令生效，机械臂开始运动 • 2：设置速度小于等于零 • 3：未初始化 • 5：轴号参数错误 • 6：伺服未开启 • 11：手机端在控制 • 101：传入参数 NOT A NUMBER • 102：发生碰撞，本次指令无效 • 103：轴发生复位，需要重新初始化，本次指令无效

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
int axis=1;
float distance=30;
```

```
float speed=100;
int ret=robot->single_joint_move(axis,distance,speed);
if (ret == 1)
{
    cout << "本次指令生效，机械臂开始运动" << endl;
}
else
{
    cout << "机械臂指令无法执行" << endl;
}
```

3.2.2.16 trail_move

功能	驱动机械臂沿一条特定的轨迹运行
注意	• move_flag 等于 false 时调用有效 • 轨迹坐标 xyzr 分别保存在四个 float 类型的数组中 • 轨迹中相邻两点的距离应为 1mm
函数定义	int trail_move(int point_number, float * x, float * y, float * z, float * r, float speed)
参数	• int point_number：总点数 • float *x：存放 x 坐标数组的首地址 • float *y：存放 y 坐标数组的首地址 • float *z：存放 z 坐标数组的首地址 • float *r：存放 r 坐标数组的首地址

	• float speed：线速度
返回	• 0：正在执行其他指令，本次指令无效 • 1：本次指令生效，机械臂开始运动 • 2：设置速度小于等于零 • 3：未初始化 • 4：轨迹起点无法到达 • 6：伺服未开启 • 11：手机端在控制 • 99：急停中 • 101：传入参数 NOT A NUMBER • 102：发生碰撞，本次指令无效 • 103：轴发生复位，需要重新初始化，本次指令无效

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
int point_number=4;
float x[4]={285.960,142.516,-74.306,389.481};
float y[4]={294.328,-122.249,-74.301,342.402};
float z[4]={0,0,0,0};
float r[4]={0,0,0,0};
float speed=100;
int ret=robot->trail_move(point_number,&x[0],&y[0],&z[0],&r[0],speed);
if (ret == 1)
{
    cout << "本次指令生效，机械臂开始运动" << endl;
}
else
{
    cout << "机械臂指令无法执行" << endl;
}
```

3.2.2.17 ~~stop_move~~(过时，不推荐使用)

功能	立即结束当前正在执行的指令
注意	无
函数定义	void stop_move()
参数	无
返回	无

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
robot->stop_move();
```

3.2.2.18 is_robot_goto_target

功能	查询机械臂是否到达目标点
注意	无
函数定义	bool is_robot_goto_target()
参数	无
返回	• false：未到达 • true：已到达

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
robot->movel_xyz(400,-30,0,0,100);
robot->wait_stop();
bool ret=robot->is_robot_goto_target();
if (ret == true)
{
    cout << "已到达" << endl;
```

```
}
else
{
    cout << "未到达" << endl;
}
```

3.2.2.19 set_allow_offset_at_target_position

功能	设定 is_robot_goto_target 的评判标准
注意	无
函数定义	void set_allow_offset_at_target_position(float x_distance float y_distance float z_distance float r_distance)
参数	• float x_distance：x 轴方向允许误差（mm） • float y_distance：y 轴方向允许误差（mm） • float z_distance：z 轴方向允许误差（mm） • float r_distance：r 轴方向允许误差（deg）
返回	无

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
float x_distance=0.01;
float y_distance=0.01;
float z_distance=0.01;
float r_distance=0.01;
robot->set_allow_offset_at_target_position(x_distance,y_distance,z_distance,r_distance)
;
robot->movel_xyz(400,-30,0,0,100);
robot->wait_stop();
```

```
bool ret=robot->is_robot_goto_target();
if (ret == true)
{
    cout << "已到达" << endl;
}
else
{
    cout << "未到达" << endl;
}
```

3.2.2.20 set_catch_or_release_accuracy

功能	设定机械臂沿 z 轴竖直向下运动时，是否到达目标点的判断标准
注意	无
函数定义	void set_catch_or_release_accuracy(float accuracy);
参数	• float accuracy：判断标准（mm）
返回	无

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
float accuracy=0.01;
robot->set_catch_or_release_accuracy(accuracy);
```

3.2.2.21 judge_in_range

功能	判断点位是否可以到达
注意	无
函数定义	bool judge_in_range(float x, float y, float z,

	float rotation)
参数	• float x：x 坐标（mm） • float y：y 坐标（mm） • float z：z 坐标（mm） • float rotation：r 坐标（deg）
返回	• false：超出范围无法到达 • true：可以到达

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
float x=320,float y=0,float z=0,float rotation=0;
bool ret=robot->judge_in_range(x, y,z,rotation);
if(ret==true)
{
    cout << "点位可以到达" << endl;
}
else
{
    cout << "点位超出范围无法到达" << endl;
}
```

3.2.2.22 ~~judge_position_gesture~~(过时，不推荐使用)

功能	判断在当前位置调用 set_position_move 以 movej 模式到达 xy 时机械臂的姿态
注意	• move_flag 等于 false 时调用有效 • 返回 true 后，isReach_after_judge，angle1_after_judge，angle2_after_judge 将会被更新

函数定义	bool judge_position_gesture(float x, float y)
参数	• float x：x 坐标（mm） • float y：y 坐标（mm）
返回	• false：机械臂正在运动，判定无效 • true：判定成功，

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
float x=320,float y=0;
bool ret=robot->judge_position_gesture(x, y);
if(ret==true)
{
    cout << "判定成功" << endl;
}
else
{
    cout << "机械臂正在运动，判定无效" << endl;
}
```

3.2.2.23 joint_home

功能	使轴回到零位
注意	无
函数定义	int joint_home(int joint_num)
参数	• int joint_num：轴号
返回	• 0：未连接

	• 1：调用成功 • 2：传入参数错误 • 3：机械臂正在初始

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
int joint_num=1;
int ret=robot->joint_home(joint_num);
if(ret==1)
{
    cout << "回零成功" << endl;
}
else
{
    cout << "回零失败" << endl;
}
```

3.2.2.24 movel_xyz

功能	以 movel 的运动方式到达目标点
注意	• move_flag 等于 false 时，进行首次调用，返回 1 后，可以重复调用该函数设定新的目标点位 • 距离当前目标点有一定距离时，再次设定新的目标，机械臂将以一定的速度通过当前目标点，而后去往新的目标点。
函数定义	int movel_xyz(float goal_x, float goal_y, float goal_z, float goal_r, float speed

	)
参数	• float goal_x：目标点 x 坐标（mm） • float goal_y：目标点 y 坐标（mm） • float goal_z：目标点 z 坐标（mm） • float goal_r：目标点 r 坐标（deg） • float speed：xyz 点的线速度（mm/s）或轴 4 的旋转角速度（deg/s）
返回	• 0：正在执行其他指令，本次指令无效 • 1：本次指令生效，机械臂开始运动 • 2：设置速度小于等于零 • 3：未初始化 • 4：过程点无法到达 • 6：伺服未开启 • 7：存在中间过程点无法以机械臂当前姿态（手系）达到 • 11：手机端在控制 • 99：急停中 • 101：传入参数 NOT A NUMBER • 102：发生碰撞，本次指令无效 • 103：轴发生复位，需要重新初始化，本次指令无效

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
float goal_x=290;
float goal_y=40;
```

```
float goal_z=0;
float goal_r=80;
float speed=100;
int ret=robot->movel_xyz(goal_x,goal_y,goal_z,goal_r,speed);
if (ret == 1)
{
    cout << "本次指令生效，机械臂开始运动" << endl;
}
else
{
    cout << "机械臂指令无法执行" << endl;
}
```

3.2.2.25 ~~movej_xyz~~(过时，推荐使用 new_movej_xyz_lr)

功能	以 movej 的运动方式到达目标点
注意	• move_flag 等于 false 时，进行首次调用，返回 1 后，可以重复调用该函数设定新的目标点位 • 距离当前目标点有一定距离时，并且当前目标点 roughly 大于 0，再次设定新的目标，机械臂将以一定的速度在当前目标点附近通过，而后去往新的目标点。
函数定义	int movej_xyz(float goal_x, float goal_y, float goal_z, float goal_r, float speed, float roughly)
参数	• float goal_x：目标点 x 坐标（mm） • float goal_y：目标点 y 坐标（mm）

	• float goal_z：目标点 z 坐标（mm） • float goal_r：目标点 r 坐标（deg） • float speed：xyz 点的线速度（mm/s）或轴 4 的旋转角速度（deg/s） • float roughly：0 先运动到前目标点，并且在当前目标点的速度等于 0，而后在去往新的目标点，1 在有新目标点时，将在当前目标点附近以最大速度通过，此参数取值范围 0 到 1，越大通过速度越快，但是通过点距离当前目标点越远。
返回	• 0：正在执行其他指令，本次指令无效 • 1：本次指令生效，机械臂开始运动 • 2：设置速度小于等于零 • 3：未初始化 • 4：目标点无法到达 • 6：伺服未开启 • 11：手机端在控制 • 101：传入参数 NOT A NUMBER • 102：发生碰撞，本次指令无效 • 103：轴发生复位，需要重新初始化，本次指令无效

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
float goal_x=290;
float goal_y=40;
float goal_z=0;
float goal_r=80;
```

```
float speed=100;
float roughly=0;
int ret=robot->movej_xyz(goal_x,goal_y,goal_z,goal_r,speed,roughly);
if (ret == 1)
{
    cout << "本次指令生效，机械臂开始运动" << endl;
}
else
{
    cout << "机械臂指令无法执行" << endl;
}
```

3.2.2.26 ~~movej_xyz_lr~~(过时，推荐使用 new_movej_xyz_lr)

功能	以 movej 的运动方式，并以指定手系达目标点
注意	• move_flag 等于 false 时，进行首次调用，返回 1 后，可以重复调用该函数设定新的目标点位 • 距离当前目标点有一定距离时，并且当前目标点 roughly 大于 0，再次设定新的目标，机械臂将以一定的速度在当前目标点附近通过，而后去往新的目标点。
函数定义	int movej_xyz_lr(float goal_x, float goal_y, float goal_z, float goal_r, float speed, float roughly, int lr)
参数	• float goal_x：目标点 x 坐标（mm） • float goal_y：目标点 y 坐标（mm）

	• float goal_z：目标点 z 坐标（mm） • float goal_r：目标点 r 坐标（deg） • float speed：xyz 点的线速度（mm/s）或轴 4 的旋转角速度（deg/s） • float roughly：0 先运动到前目标点，并且在当前目标点的速度等于 0，而后在去往新的目标点，1 在有新目标点时，将在当前目标点附近以最大速度通过，此参数取值范围 0 到 1，越大通过速度越快，但是通过点距离当前目标点越远。 • int lr：1 右手系，-1 左手系
返回	• 0：正在执行其他指令，本次指令无效 • 1：本次指令生效，机械臂开始运动 • 2：设置速度小于等于零 • 3：未初始化 • 4：目标点无法到达 • 6：伺服未开启 • 7：无法以设定手系到达目标点位 • 11：手机端在控制 • 101：传入参数 NOT A NUMBER • 102：发生碰撞，本次指令无效 • 103：轴发生复位，需要重新初始化，本次指令无效

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
```

```
float goal_x=290;
float goal_y=40;
float goal_z=0;
float goal_r=80;
float speed=100;
float roughly=0;
int lr=1;
int ret=robot->movej_xyz_lr(goal_x,goal_y,goal_z,goal_r,speed,roughly,lr);
if (ret == 1)
{
    cout << "本次指令生效，机械臂开始运动" << endl;
}
else
{
    cout << "机械臂指令无法执行" << endl;
}
```

3.2.2.27 ~~movej_angle~~(过时，推荐使用 new_movej_xyz_lr)

功能	以 movej 的运动方式到达目标点
注意	• move_flag 等于 false 时，进行首次调用，返回 1 后，可以重复调用该函数设定新的目标点位 • 距离当前目标点有一定距离时，并且当前目标点 roughly 大于 0，再次设定新的目标，机械臂将以一定的速度在当前目标点附近通过，而后去往新的目标点。
函数定义	int movej_angle(float angle1, float angle2, float goal_z, float goal_r, float speed, float roughly)

参数	• float angle1：目标点轴 1 角度坐标（deg） • float angle2：目标点轴 2 角度坐标（deg） • float goal_z：目标点 z 坐标（mm） • float goal_r：目标点 r 坐标（deg） • float speed：xyz 点的线速度（mm/s）或轴 4 的旋转角速度（deg/s） • float roughly：0 先运动到前目标点，并且在当前目标点的速度等于 0，而后在去往新的目标点，1 在有新目标点时，将在当前目标点附近以最大速度通过，此参数取值范围 0 到 1，越大通过速度越快，但是通过点距离当前目标点越远。
返回	• 0：正在执行其他指令，本次指令无效 • 1：本次指令生效，机械臂开始运动 • 2：设置速度小于等于零 • 3：未初始化 • 4：目标点无法到达 • 6：伺服未开启 • 11：手机端在控制 • 101：传入参数 NOT A NUMBER • 102：发生碰撞，本次指令无效 • 103：轴发生复位，需要重新初始化，本次指令无效

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
```

```
float angle1=60;
float angle2=40;
float goal_z=0;
float goal_r=80;
float speed=100;
float roughly=0;
int ret=robot->movej_angle(angle1,angle2,goal_z,goal_r,speed,roughly);
if (ret == 1)
{
    cout << "本次指令生效，机械臂开始运动" << endl;
}
else
{
    cout << "机械臂指令无法执行" << endl;
}
```

3.2.2.28 change_attitude

功能	切换手系
注意	
函数定义	int change_attitude(float speed)
参数	float speed：线速度（mm/s）
返回	• 0：机械臂正在运行其他指令，本次指令无效 • 1：本次指令生效，机械臂开始移动 • 2：传入的速度小于等于 0 • 3：尚未初始化 • 4：另一姿态无法到达 • 6：伺服未开启 • 11： 手机端正在控制 • 101： 传入参数 NAN

	• 102： 发生碰撞，无法运动 • 103： 关节发生复位，需要重新初始化

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
float speed=100;
int ret=robot->change_attitude(speed);
if (ret == 1)
{
    cout << "本次指令生效，机械臂开始运动" << endl;
}
else
{
    cout << "机械臂指令无法执行" << endl;
}
```

3.2.2.29 wait_stop

功能	用于判断运动是否结束
注意	• 阻塞函数，直到运动结束才会返回
函数定义	bool wait_stop()
参数	无
返回	• true：正常停止 • false：存在某些异常使 wait_stop 超时,此时应停止手臂作业流程

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
bool ret=robot->wait_stop();
if (ret == true)
{
    cout << "正常停止" << endl;
```

```
}
else
{
    cout << "存在某些异常使wait_stop超时,此时应停止手臂作业流程" << endl;
}
```

3.2.2.30 pause_move

功能	暂停机械臂的运动
注意	函数返回后，经过一定延时（几百毫秒），机械臂停止运动
函数定义	int pause_move()
参数	无
返回	• 默认返回 1

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
return robot->pause_move();
```

3.2.2.31 resume_move

功能	恢复机械臂的运动
注意	无
函数定义	int resume_move()
参数	无
返回	• 默认返回 1

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
```

```
return robot->pause_move();
```

3.2.2.32 set_digital_out

功能	设定 io 输出点状态
注意	
函数定义	bool set_digital_out(int io_number, bool value)
参数	• int io_number：io 输出点编号 • bool value：设定值
返回	• 0：io_number 参数错误 • 1：设置成功 • 3：未初始化;

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
int io_number=1;
bool value=true;
bool ret=robot->set_digital_out(io_number,value);
if (ret == 1)
{
    cout << "设置成功" << endl;
}
else
{
    cout << "设置失败" << endl;
}
```

3.2.2.33 get_digital_out

功能	获取 io 输出点状态
注意	
函数定义	int get_digital_out(int io_out_num)
参数	int io_out_num：io 输出点编号
返回	• -1：io_out_num 参数错误 • 0：io 口输出状态为断开 • 1：io 口输出状态为导通 • 3：未初始化

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
int io_out_num=1;
int ret=robot->get_digital_out(io_out_num);
if (ret == 1)
{
    cout << "io口输出状态为导通" << endl;
}
else
{
    cout << "io口输出状态为断开" << endl;
}
```

3.2.2.34 get_digital_in

功能	获取 io 输入点状态
注意	
函数定义	int get_digital_in(int io_in_number)

参数	int io_in_number：io 输入点编号
返回	• -1：io_in_number 参数错误 • 0：io 输入点没有被触发 • 1：io 输入点被触发 • 3：未初始化

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
int io_in_number=1;
int ret=robot->get_digital_in(io_in_number);
if (ret == 1)
{
    cout << "io输入点被触发" << endl;
}
else
{
    cout << "io 输入点没有被触发" << endl;
}
```

3.2.2.35 set_efg_state

功能	设定电动夹爪状态
注意	•切换控制类型需要重启机械臂和 sdk
函数定义	int set_efg_state(int type, float distance)
参数	• int type：8 EFG-8，20 EFG-20 • float distance： Type==8：

	0 张开 1 闭合 Type==20：实际行程（mm）
返回	• -1：控制参数发生变化 • 0：type 参数错误 • 1：设置成功 • 3：未初始化;

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
int type=20;
float distance=20;
int ret=robot->set_efg_state(type,distance);
if (ret == 1)
{
    cout << "设置成功" << endl;
}
else
{
    cout << "设置失败" << endl;
}
```

3.2.2.36 get_efg_state

功能	获取电动夹爪状态
注意	• 1632 机型不支持在运动过程中调用
函数定义	int get_efg_state (int* type, float* distance)

参数	• int* type：电爪类型指针 • float* distance：电爪当前状态指针
返回	• 1：成功 • 3：失败，未初始化;

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
int type=20;
float distance=0;
int ret=robot->get_efg_state(&type,&distance);
if (ret == 1)
{
    cout << "获取成功" << endl;
}
else
{
    cout << "获取失败" << endl;
}
```

3.2.2.37 new_movej_xyz_lr

功能	以 movej 的运动方式，并以指定手系达目标点
注意	• move_flag 等于 false 时，进行首次调用，返回 1 后，可以重复调用该函数设定新的目标点位 • 距离当前目标点有一定距离时，并且当前目标点 roughly 大于 0，再次设定新的目标，机械臂将以一定的速度在当前目标点附近通过，而后去往新的目标点。
函数定义	int new_movej_xyz_lr(float goal_x, float goal_y,

	float goal_z, float goal_r, float speed, float roughly, int lr)
参数	• float goal_x：目标点 x 坐标（mm） • float goal_y：目标点 y 坐标（mm） • float goal_z：目标点 z 坐标（mm） • float goal_r：目标点 r 坐标（deg） • float speed：xyz 点的线速度（mm/s）或轴 4 的旋转角速度（deg/s） • float roughly：0 先运动到前目标点，并且在当前目标点的速度等于 0，而后在去往新的目标点，1 在有新目标点时，将在当前目标点附近以最大速度通过，此参数取值范围 0 到 1，越大通过速度越快，但是通过点距离当前目标点越远。 • int lr：1 右手系，-1 左手洗，angle2>0 右手系，否则为左手系
返回	• 0：正在执行其他指令，本次指令无效 • 1：本次指令生效，机械臂开始运动 • 2：设置速度小于等于零 • 3：未初始化 • 4：目标点无法到达

	• 6：伺服未开启 • 7：无法以设定手系到达目标点位 • 11：手机端在控制 • 99：急停中 • 101：传入参数 NOT A NUMBER • 102：发生碰撞，本次指令无效 • 103：轴发生复位，需要重新初始化，本次指令无效

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
float goal_x=290;
float goal_y=40;
float goal_z=0;
float goal_r=80;
float speed=100;
float roughly=0;
int lr=1;
int ret=robot->new_movej_xyz_lr(goal_x,goal_y,goal_z,goal_r,speed,roughly,lr);
if (ret == 1)
{
    cout << "本次指令生效，机械臂开始运动" << endl;
}
else
{
    cout << "机械臂指令无法执行" << endl;
}
```

3.2.2.38 new_set_acc

功能	设定 new_movej_xyz_lr 运动模型的关节加速度百分比
注意	无，范围>=30,<=220

函数定义	int new_set_acc(int j1_max_acc, int j2_max_acc, int j3_max_acc, int j4_max_acc)
参数	• int j1_max_acc：关节 1 加速度百分比 • int j1_max_acc：关节 2 加速度百分比 • int j1_max_acc：关节 3 加速度百分比 • int j1_max_acc：关节 4 加速度百分比
返回	• 1：默认返回 1

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
int j1_max_acc=120;
int j2_max_acc=120;
int j3_max_acc=120;
int j4_max_acc=120;
return robot->new_set_acc(j1_max_acc, j2_max_acc, j3_max_acc, j4_max_acc);
```

3.2.2.39 j5_motor_zero

功能	1）搭配步进电机扩展模块使用，调用后会将当前位置作为零位 2）搭配第五轴使用时，无效
注意	无
函数定义	int j5_motor_zero()
参数	无

返回	• 0：手臂未连接 • 1：设定成功 • 3：手臂未初始化 • 4：不支持该功能

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
int ret=robot->j5_motor_zero();
if (ret == 1)
{
    cout << "设定成功" << endl;
}
else
{
    cout << "设定失败" << endl;
}
```

3.2.2.40 set_j5_motor_pos

功能	1）搭配步进电机扩展模块使用，设定步进电机的目前位置，范围限制正负 1000 圈 2）搭配第五轴使用时，范围正负 150 度
注意	手臂本体四个关节在运动时，禁止调用此接口
函数定义	int set_j5_motor_pos(float deg, float speed);
参数	• float deg：目标角度 单位 deg • float speed：平均速度 单位 deg/s
返回	• 0：其他运动函数调用中 • 1：设定成功

	• 2：speed 小于等于零 • 3：未初始化 • 4：手臂当前 xyzr 超出运动范围 • 99：急停中 • 102：手臂发生碰撞 • 103：关节复位 • 105：关节异常 • 106：不支持该功能

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
float deg=60;
float speed=100;
int ret=robot->set_j5_motor_pos(deg,speed);
if (ret == 1)
{
    cout << "设定成功" << endl;
}
else
{
    cout << "设定失败" << endl;
}
```

3.2.2.41 get_j5_parameter

功能	搭配步进电机扩展模块或第五轴使用，返回电机电机当前位置
注意	无
函数定义	int get_j5_parameter();

参数	无
返回	• 返回五轴当前位置

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
return robot->get_j5_parameter();
```

3.2.2.42 movej_j5

功能	设定第五轴舵机位置
注意	• 1632 机型不支持
函数定义	int movej_j5(float j5_pos, float speed);
参数	• float j5_pos,：舵机位置 deg • float speed：舵机转速 deg/s
返回	• 0：正在执行其他指令，本次指令无效 • 1：本次指令生效，机械臂开始运动 • 2：设置速度小于等于零 • 3：未初始化 • 6：伺服未开启 • 11：手机端在控制 • 101：传入参数 NOT A NUMBER • 102：发生碰撞，本次指令无效

	• 103：轴发生复位，需要重新初始化，本次指令无效

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
float j5_pos=60;
float speed=100;
int ret=robot->movej_j5(j5_pos,speed);
if (ret == 1)
{
    cout << "本次指令生效，机械臂开始运动" << endl;
}
else
{
    cout << "机械臂指令无法执行" << endl;
}
```

3.2.2.43 ~~get_j5_positon~~（过时，建议使用 get_j5_paramter)

功能	获取第五轴舵机的位置
注意	• 1632 机型不支持
函数定义	float get_j5_positon(int type);
参数	• int type：舵机传入 2
返回	•返回舵机位置 deg

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
int type=2;
return robot->get_j5_positon(type);
```

3.2.2.44 get_efg_state_dji

功能	搭配 EFG-NMA 电动夹爪上电夹指不动版本使用，用于查询

	夹爪位置
注意	仅用于控制上电夹指不动 EFG-NMA 电动夹爪，上电位置为 0
函数定义	int get_efg_state_dji (int* type, float* distance)
参数	• int* type：8:EFG-8 20:EFG-20NMA 50:EFG-50NMA • float* distance：type==8：0 张开 1 闭合 type==20：实际行程（mm），范围 0~20 type==50：实际行程（mm），范围 0~50
返回	• 1：查询夹爪位置成功 • 2：连接未初始化 • 3：未初始化

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
int type=20;
float distance=0;
int ret=robot->get_efg_state_dji(&type,&distance);
if (ret == 1)
{
    cout << "查询夹爪位置成功" << endl;
}
else
{
    cout << "查询夹爪位置失败" << endl;
}
```

3.2.2.45 set_efg_state_dji

功能	搭配 EFG-NMA 电动夹爪上电夹指不动版本使用，用于控制夹爪移动
注意	仅用于控制上电夹指不动 EFG-NMA 电动夹爪，上电位置为 0，当电动夹爪是 EFG-20NMA 或 EFG-50NMA 时，需要进行上电校准操作，EFG-20NMA 执行 set_efg_state_dji (20, -1000)上电校准操作，EFG-50NMA 执行 set_efg_state_dji (50, -1000)上电校准操作
函数定义	int set_efg_state_dji(int type, float distance)
参数	• int type：8:EFG-8 20:EFG-20NMA 50:EFG-50NMA • float distance：type==8：0 张开 1 闭合 type==20：实际行程（mm），范围 0~20 type==50：实际行程（mm），范围 0~50
返回	• -10：急停中 • -1：控制参数发生变化 • 0：type 参数错误 • 1：设置成功 • 2: 1235 端口长时间被占用超时或者 1235 端口查询线程挂掉超时

	• 3：未初始化 • 4：50nma 夹爪控制机型错误 • 5：爪子运行出错 • 101：传入参数 NOT A NUMBER

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
int type=20;
float distance=0;
int ret=robot->set_efg_state_dji(type,distance);
if (ret == 1)
{
    cout << "设置成功" << endl;
}
else
{
    cout << "设置失败" << endl;
}
```

3.2.2.46 new_stop_move

功能	立即结束当前正在执行的指令
注意	相比 stop_move，优化了停止算法，建议搭配 new_movej_xyz_lr 使用
函数定义	void new_stop_move()
参数	无
返回	无

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
```

```
robot->new_stop_move();
```

3.2.2.47 new_movej_angle

功能	以 movej 的运动方式到达目标点
注意	• move_flag 等于 false 时，进行首次调用，返回 1 后，可以重复调用该函数设定新的目标点位 • 距离当前目标点有一定距离时，并且当前目标点 roughly 大于 0，再次设定新的目标，机械臂将以一定的速度在当前目标点附近通过，而后去往新的目标点。
函数定义	int new_movej_angle(float angle1, float angle2, float goal_z, float goal_r, float speed, float roughly)
参数	• float angle1：目标点轴 1 角度坐标（deg） • float angle2：目标点轴 2 角度坐标（deg） • float goal_z：目标点 z 坐标（mm） • float goal_r：目标点 r 坐标（deg） • float speed：xyz 点的线速度（mm/s）或轴 4 的旋转角速度（deg/s） • float roughly：0 先运动到前目标点，并且在当前目标点的速度等于 0，而后在去往新的目标点，1 在有新目标点时，将

	在当前目标点附近以最大速度通过，此参数取值范围 0 或 1
返回	• 0：正在执行其他指令，本次指令无效 • 1：本次指令生效，机械臂开始运动 • 2：设置速度小于等于零 • 3：未初始化 • 4：目标点无法到达 • 6：伺服未开启 • 11：手机端在控制 • 99：急停中 • 101：传入参数 NOT A NUMBER • 102：发生碰撞，本次指令无效 • 103：轴发生复位，需要重新初始化，本次指令无效

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
float angle1=60;
float angle2=40;
float goal_z=0;
float goal_r=80;
float speed=100;
float roughly=0;
int ret=robot->new_movej_angle(angle1,angle2,goal_z,goal_r,speed,roughly);
if (ret == 1)
{
    cout << "本次指令生效，机械臂开始运动" << endl;
}
else
{
    cout << "机械臂指令无法执行" << endl;
}
```

3.2.2.48 new_set_efg_state(新增双通道)

功能	设定电动夹爪状态
注意	•切换控制类型需要重启机械臂和 sdk,当电动夹爪是 EFG-20NMA、EFG-30NMA 或 EFG-50NMA 时，需要进行上电校准操作，EFG-20NMA 执行 new_set_efg_state (channel,-20, -1000) 上电校准操作，EFG-30NMA 执行 new_set_efg_state (channel,-30, -1000)上电校准操作，EFG-50NMA 执行 new_set_efg_state (channel,-50, -1000)上电校准操作
函数定义	int new_set_efg_state(int channel int type, float distance)
参数	• int channel：0: 通道 0，1: 通道 1， • int type：8: EFG-8， 20: EFG-20NM -20:EFG-20NMA 30:EFG-30NM -30:EFG-30NMA 50:EFG-50NM -50:50NMA • float distance： Type==8： 0 张开 1 闭合

	Type==20，-20，30，-30，50，-50：实际行程（mm）
返回	• -10：急停中 • -1：该机型不支持 • 0：type 参数错误 • 1：设置指令成功 • 2：输入参数错误 • 3：未初始化 • 101：参数异常，被篡改

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
int channel=0;
int type=20;
float distance=0;
int ret=robot->new_set_efg_state(channel,type,distance);
if (ret == 1)
{
    cout << "设置成功" << endl;
}
else
{
    cout << "设置失败" << endl;
}
```

3.2.2.49 new_get_efg_state(新增双通道)

功能	获取电动夹爪状态
注意	
函数定义	int new_get_efg_state (int channel

	int type, float* distance)
参数	• int channel : 0: 通道 0，1: 通道 1， • int type：电爪类型 • float* distance：电爪当前状态指针
返回	• -1：该机型不支持； • 1：获取成功； • 2：输入参数错误； • 3：未初始化；

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
int channel=0;
int type=20;
float distance=0;
int ret=robot->new_get_efg_state(channel,type,&distance);
if (ret == 1)
{
    cout << "获取成功" << endl;
}
else
{
    cout << "获取失败" << endl;
}
```

3.2.2.50 get_hard_emergency_stop_state

功能	获取急停状态
注意	目前 4150,4160 新机型支持

函数定义	int get_hard_emergency_stop_state()
参数	无
返回	-1：该机型不支持； 0：未处于急停状态； 1：处于急停状态； 3：未连接； 4：没有急停功能；

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
int ret=robot->get_hard_emergency_stop_state();
if (ret == 1)
{
    cout << "处于急停状态" << endl;
}
else
{
    cout << "未处于急停状态" << endl;
}
```

3.2.2.51 clear_hard_emergency_stop

功能	急停按钮复位后，发送清除急停状态，进入正常模式
注意	目前 4150,4160 新机型支持
函数定义	int clear_hard_emergency_stop()
参数	无
返回	-1：该机型不支持； 0：未处于急停状态；

	1：解除成功； 2：解除超时 3：未连接； 4：没有急停功能； 5：正处于急停状态

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
int ret=robot->clear_hard_emergency_stop();
if (ret == 1)
{
    cout << "解除成功" << endl;
}
else
{
    cout << "解除失败" << endl;
}
```

3.2.2.52 set_io_card_out_type

功能	设置新版末端 IO 板输出类型
注意	目前仅 4150,4160 新机型支持
函数定义	int set_io_card_out_type (int type)
参数	• int type: 0: PNP ， 1: NPN，
返回	-1：该机型不支持； 1：设置成功； 2：设置超时 3：未初始化；

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
int type=0;
int ret=robot->set_io_card_out_type(type);
if (ret == 1)
{
    cout << "设置成功" << endl;
}
else
{
    cout << "设置失败" << endl;
}
```

3.2.2.53 set_brake_state

功能	控制抱闸开合
注意	无
函数定义	int set_brake_state (int priority, int state)
参数	• int priority : 0: 非强制控制抱闸方式，1: 强制控制抱闸方式 • int state:0: 吸合，1: 打开
返回	1：设置成功； 2：初始化未连接； 3：运动过程中不循序闭合抱闸；

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
int priority=0;
int state=0;
int ret=robot->set_brake_state(priority,state);
if (ret == 1)
{
```

```
    cout << "设置成功" << endl;
}
else
{
    cout << "设置失败" << endl;
}
```

3.2.2.54 get_brake_state

功能	获取抱闸开合状态
注意	无
函数定义	int get_brake_state ()
参数	无
返回	0：闭合状态； 1：张开状态； 3：初始化未连接；

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
return robot->get_brake_state();
```

3.2.2.55 set_tool_fun1

功能	通过工具长度和工具角度设置工具坐标
注意	无
函数定义	int set_tool_fun1 (float Tool_L,float Tool_R)
参数	• float Tool_L：工具长度 • float Tool_R: 工具角度

返回	1：设置工具坐标成功； 2：设置工具长度小于 0； 3：运动过程中不可设置工具坐标；

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
float Tool_L=20;
float Tool_R=0;
int ret=robot->set_tool_fun1(Tool_L,Tool_R);
if (ret == 1)
{
    cout << "设置工具坐标成功" << endl;
}
else
{
    cout << "设置工具坐标失败" << endl;
}
```

3.2.2.56 set_tool_fun2

功能	通过工具中心点设置工具坐标
注意	无
函数定义	int set_tool_fun2 (float p_x,float p_y)
参数	• float p_x：工具中心点 x 坐标 • float p_y: 工具中心点 y 坐标
返回	1：设置工具坐标成功； 2：设置工具长度小于 0； 3：运动过程中不可设置工具坐标；

代码示例:

```
int robot_id = 84;
```

```
ControlBeanEx *robot;
robot = get_robot(robot_id);
float p_x=20;
float p_y=0;
int ret=robot->set_tool_fun2(p_x,p_y);
if (ret == 1)
{
    cout << "设置工具坐标成功" << endl;
}
else
{
    cout << "设置工具坐标失败" << endl;
}
```

3.2.2.57 set_tool_fun3

功能	通过两点法设置工具坐标
注意	无
函数定义	int set_tool_fun3 (float p1_x,float p1_y, float p1_r,float p2_x, float p2_y,float p2_r)
参数	• float p1_x：第 1 点 x 坐标 • float p1_y：第 1 点 y 坐标 • float p1_r：第 1 点 r 坐标 • float p2_x：第 2 点 x 坐标 • float p2_y：第 2 点 y 坐标 • float p2_r：第 2 点 r 坐标
返回	1：设置工具坐标成功； 2：设置工具长度小于 0； 3：运动过程中不可设置工具坐标；

代码示例:

```
int robot_id = 84;
```

```
ControlBeanEx *robot;
robot = get_robot(robot_id);
float p1_x=249.3410;
float p1_y=158.8480;
float p1_r=0;
float p2_x=264.0866;
float p2_y=172.3598;
float p2_r=0;
int ret=robot->set_tool_fun3(p1_x,p1_y,p1_r,p2_x,p2_y,p2_r);
if (ret == 1)
{
    cout << "设置工具坐标成功" << endl;
}
else
{
    cout << "设置工具坐标失败" << endl;
}
```

3.2.2.58 com485_initial

功能	RS485 夹爪初始化设置波特率
注意	无
函数定义	int com485_initial (int baudRate)
参数	• int baudRate：波特率
返回	1：夹爪初始化成功； 2：未初始化； 3：未连接； 4：波特率过小； 5：波特率过大； 6：接收超时； 7：设定反馈超时；

	8：非 485 扩展模块； -1：I/O 板发送超时；

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
int baudRate=115200;
int ret=robot->com485_initial(baudRate);
if (ret == 1)
{
    cout << "RS485夹爪初始化设置波特率成功" << endl;
}
else
{
    cout << "RS485 夹爪初始化设置波特率失败" << endl;
}
```

3.2.2.59 com485_send

功能	RS485 夹爪发送数据控制夹爪
注意	无
函数定义	int com485_send (unsigned char *data,unsigned char len)
参数	• unsigned char *data : 485 夹爪数据 • unsigned char len : 485 夹爪数据长度
返回	1：夹爪发送数据成功； -1：I/O 板发送超时； -2：485 发送超时； 2：准备发送状态位置超时； 3：下位机进入接收发送数据状态超时；

	6：接收超时； 8：非 485 扩展模块；

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
unsigned char  data_open[13] = { 0x01, 0x10, 0x00, 0x02, 0x00, 0x02, 0x04, 0x41, 0xA0, 0x00,
0x00, 0x66, 0x68 };//夹爪张开20mm
int ret=robot->com485_send(&data_open[0],13);
if (ret == 1)
{
    cout << "RS485夹爪发送数据成功" << endl;
}
else
{
    cout << "RS485 夹爪发送数据失败" << endl;
}
```

3.2.2.60 com485_recv

功能	RS485 夹爪接收返回数据
注意	无
函数定义	int com485_recv (unsigned char *data)
参数	• unsigned char *data : 485 夹爪返回数据
返回	8：夹爪返回数据长度； -1：非 485 扩展模块； 0：485 接收失败；

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
unsigned char  data_recv[13] {0};
int ret=robot->com485_recv(&data_recv[0]);
```

```
if (ret > 0)
{
    for (int i = 0; i < ret; i++)
    {
        cout << hex << int(data_recv[i]) << " ";
        cout << "\n" << endl;
    }
}
```

3.2.2.61 set_user

功能	设置用户坐标系
注意	无
函数定义	int set_user(float x0,float y0,float x1,float y1)
参数	• float x0：用户坐标系原点 x 坐标 • float y0：用户坐标系原点 y 坐标 • float x1：用户坐标系 x 轴方向上的点 x 坐标 • float y1：用户坐标系 x 轴方向上的点 y 坐标
返回	1：用户坐标系设置成功； 2：原点和 x 轴方向上的点重合； 3：运动过程中不可设置用户坐标系； 101：点位数据非数值 NaN；

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
float x0=249.3410;
float y0=158.8480;
float x1=264.0866;
float y1=172.3598;
int ret=robot->set_user(x0,y0,x1,y1);
if (ret == 1)
```

```
{
    cout << "设置用户坐标成功" << endl;
}
else
{
    cout << "设置用户坐标失败" << endl;
}
```

3.1.2.62 com485_init

功能	485 夹爪手动初始化
注意	无
函数定义	int com485_init()
参数	无
返回	1：夹爪手动初始化成功； -1：I/0 板发送超时； -2：485 发送超时； 2：准备发送状态位置超时； 3：下位机进入接收发送数据状态超时； 6：接收超时； 8：非 485 扩展模块；

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
int ret=robot->com485_init();
if (ret == 1)
{
    cout << "RS485夹爪手动初始化成功" << endl;
}
else
{
```

```
    cout << "RS485 夹爪手动初始化失败" << endl;
}
```

3.1.2.63 com485_set_rotation_speed

功能	485 夹爪设置旋转速度
注意	无
函数定义	int com485_set_rotation_speed(float speed)
参数	• float speed：旋转速度
返回	1：夹爪设置旋转速度成功； -1：I/0 板发送超时； -2：485 发送超时； 2：准备发送状态位置超时； 3：下位机进入接收发送数据状态超时； 6：接收超时； 8：非 485 扩展模块；

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
int baudRate=115200;
robot->com485_initial(baudRate);
int ret=robot->com485_set_rotation_speed(50);
if (ret == 1)
{
    cout << "RS485夹爪设置旋转速度成功" << endl;
}
else
{
    cout << "RS485 夹爪设置旋转速度失败" << endl;
```

```
}
```

3.1.2.64 com485_set_clamping_speed

功能	485 夹爪设置夹持速度
注意	无
函数定义	int com485_set_clamping_speed(float speed)
参数	• float speed：夹持速度
返回	1：夹爪设置夹持速度成功； -1：I/0 板发送超时； -2：485 发送超时； 2：准备发送状态位置超时； 3：下位机进入接收发送数据状态超时； 6：接收超时； 8：非 485 扩展模块；

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
int baudRate=115200;
robot->com485_initial(baudRate);
int ret=robot->com485_set_clamping_speed(50);
if (ret == 1)
{
    cout << "RS485夹爪设置夹持速度成功" << endl;
}
else
{
    cout << "RS485 夹爪设置夹持速度失败" << endl;
}
```

3.1.2.65 com485_set_clamping_distance

功能	485 夹爪设置夹持距离
注意	无
函数定义	int com485_set_clamping_distance(float distance)
参数	• float distance：夹持距离
返回	1：夹爪设置夹持距离成功； -1：I/0 板发送超时； -2：485 发送超时； 2：准备发送状态位置超时； 3：下位机进入接收发送数据状态超时； 6：接收超时； 8：非 485 扩展模块；

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
int baudRate=115200;
robot->com485_initial(baudRate);
int ret=robot->com485_set_clamping_distance(10);
if (ret == 1)
{
    cout << "RS485夹爪设置夹持距离成功" << endl;
}
else
{
    cout << "RS485 夹爪设置夹持距离失败" << endl;
}
```

3.1.2.66 com485_get_clamping_distance

功能	485 夹爪获取夹持距离
注意	无
函数定义	float com485_get_clamping_distance()
参数	• 无
返回	夹持距离

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
int baudRate=115200;
robot->com485_initial(baudRate);
return robot->com485_get_clamping_distance();
```

3.1.2.67 com485_set_rotation_angle

功能	485 夹爪设置绝对旋转角度
注意	无
函数定义	int com485_set_rotation_angle(float angle)
参数	• float angle：绝对旋转角度
返回	1：夹爪设置绝对旋转角度成功； -1：I/0 板发送超时； -2：485 发送超时； 2：准备发送状态位置超时； 3：下位机进入接收发送数据状态超时；

	6：接收超时； 8：非 485 扩展模块；

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
int baudRate=115200;
robot->com485_initial(baudRate);
int ret=robot->com485_set_rotation_angle(10);
if (ret == 1)
{
    cout << "RS485夹爪设置绝对旋转角度成功" << endl;
}
else
{
    cout << "RS485 夹爪设置绝对旋转角度失败" << endl;
}
```

3.1.2.68 com485_set_relative_rotation_angle

功能	485 夹爪设置相对旋转角度
注意	无
函数定义	int com485_set_relative_rotation_angle(float angle)
参数	• float angle：相对旋转角度
返回	1：夹爪设置相对旋转角度成功； -1：I/0 板发送超时； -2：485 发送超时； 2：准备发送状态位置超时； 3：下位机进入接收发送数据状态超时； 6：接收超时；

	8：非 485 扩展模块；

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
int baudRate=115200;
robot->com485_initial(baudRate);
int ret=robot->com485_set_relative_rotation_angle(10);
if (ret == 1)
{
    cout << "RS485夹爪设置相对旋转角度成功" << endl;
}
else
{
    cout << "RS485 夹爪设置相对旋转角度失败" << endl;
}
```

3.1.2.69 com485_get_rotation_angle

功能	485 夹爪获取旋转角度
注意	无
函数定义	float com485_get_rotation_angle()
参数	无
返回	旋转角度

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
int baudRate=115200;
robot->com485_initial(baudRate);
return robot->com485_get_rotation_angle();
```

3.1.2.70 com485_get_clamping_state

功能	485 夹爪获取夹持状态

注意	无
函数定义	int com485_get_clamping_state()
参数	无
返回	0：到位； 1：运动中； 2：夹持； 3：掉落；

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
int baudRate=115200;
robot->com485_initial(baudRate);
return robot->com485_get_clamping_state();
```

3.1.2.71 com485_get_rotation_state

功能	485 夹爪获取旋转状态
注意	无
函数定义	int com485_get_rotation_state()
参数	无
返回	0：到位； 1：旋转中； 2：旋转受阻； 3：掉落； 4：堵转停转；

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
int baudRate=115200;
robot->com485_initial(baudRate);
return robot->com485_get_rotation_state();
```

3.1.2.72 com485_set_clamping_current

功能	485 夹爪设置夹持电流
注意	无
函数定义	int com485_set_clamping_current(float current)
参数	• float current：夹持电流
返回	1：夹爪设置夹持电流成功； -1：I/0 板发送超时； -2：485 发送超时； 2：准备发送状态位置超时； 3：下位机进入接收发送数据状态超时； 6：接收超时； 8：非 485 扩展模块；

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
int baudRate=115200;
robot->com485_initial(baudRate);
int ret=robot->com485_set_clamping_current(0.2);
if (ret == 1)
{
    cout << "RS485夹爪设置夹持电流成功" << endl;
}
```

```
else
{
    cout << "RS485 夹爪设置夹持电流失败" << endl;
}
```

3.1.2.73 com485_set_rotation_current

功能	485 夹爪设置旋转电流
注意	无
函数定义	int com485_set_rotation_current(float current)
参数	• float current：旋转电流
返回	1：夹爪设置旋转电流成功； -1：I/0 板发送超时； -2：485 发送超时； 2：准备发送状态位置超时； 3：下位机进入接收发送数据状态超时； 6：接收超时； 8：非 485 扩展模块；

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
int baudRate=115200;
robot->com485_initial(baudRate);
int ret=robot->com485_set_rotation_current(0.2);
if (ret == 1)
{
    cout << "RS485夹爪设置旋转电流成功" << endl;
}
else
{
    cout << "RS485 夹爪设置旋转电流失败" << endl;
```

```
}
```

3.1.2.74 com485_get_clamping_current

功能	485 夹爪获取夹持电流
注意	无
函数定义	float com485_get_clamping_current()
参数	无
返回	夹持电流

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
int baudRate=115200;
robot->com485_initial(baudRate);
return robot->com485_get_clamping_current();
```

3.1.2.75 com485_get_rotation_current

功能	485 夹爪获取旋转电流
注意	无
函数定义	float com485_get_rotation_current()
参数	无
返回	旋转电流

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
int baudRate=115200;
robot->com485_initial(baudRate);
return robot->com485_get_rotation_current();
```

3.1.2.76 com485_suction_cup_control

功能	485 吸盘控制
注意	无
函数定义	int com485_suction_cup_control(int type)
参数	• int type：控制类型 1：吸取 2：释放 3：按真空度设置吸取
返回	1：吸盘控制成功； -1：I/0 板发送超时； -2：485 发送超时； 2：准备发送状态位置超时； 3：下位机进入接收发送数据状态超时； 6：接收超时； 8：非 485 扩展模块；

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
int baudRate=115200;
robot->com485_initial(baudRate);
int ret=robot->com485_suction_cup_control(1);
if (ret == 1)
{
    cout << "RS485吸盘吸取成功" << endl;
}
else
{
    cout << "RS485 吸盘吸取失败" << endl;
}
```

3.1.2.77 com485_min_vacuum_degree

功能	485 吸盘最小真空度设置

注意	无
函数定义	int com485_min_vacuum_degree(int degree)
参数	• int degree：真空度
返回	1：吸盘最小真空度设置成功； -1：I/0 板发送超时； -2：485 发送超时； 2：准备发送状态位置超时； 3：下位机进入接收发送数据状态超时； 6：接收超时； 8：非 485 扩展模块；

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
int baudRate=115200;
robot->com485_initial(baudRate);
int ret=robot->com485_min_vacuum_degree(10);
if (ret == 1)
{
    cout << "RS485吸盘最小真空度设置成功" << endl;
}
else
{
    cout << "RS485 吸盘最小真空度设置失败" << endl;
}
```

3.1.2.78 com485_max_vacuum_degree

功能	485 吸盘最大真空度设置
注意	无

函数定义	int com485_max_vacuum_degree(int degree)
参数	• int degree：真空度
返回	1：吸盘最大真空度设置成功； -1：I/0 板发送超时； -2：485 发送超时； 2：准备发送状态位置超时； 3：下位机进入接收发送数据状态超时； 6：接收超时； 8：非 485 扩展模块；

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
int baudRate=115200;
robot->com485_initial(baudRate);
int ret=robot->com485_max_vacuum_degree(50);
if (ret == 1)
{
    cout << "RS485吸盘最大真空度设置成功" << endl;
}
else
{
    cout << "RS485 吸盘最大真空度设置失败" << endl;
}
```

3.1.2.79 com485_get_draw_state

功能	485 吸盘获取吸取状态
注意	无
函数定义	int com485_get_draw_state()

参数	无
返回	0：初始化或者释放物体； 1：吸住物体； 2：未吸住物体或者物体掉落；

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
int baudRate=115200;
robot->com485_initial(baudRate);
return robot->com485_get_draw_state();
```

3.1.2.80 get_error_code

功能	获取错误信息
注意	无
函数定义	int get_error_code()
参数	无
返回	错误信息，详细见错误表

代码示例:

```
int robot_id = 84;
ControlBeanEx *robot;
robot = get_robot(robot_id);
return robot->get_error_code();
```

慧灵科技(深圳)有限公司
Huiling-tech Robotic Co.,Ltd

电话 0755-36382405
邮箱 hitbot@hitbot.cc
网址 www.hitbot.cc
地址 广东省深圳市宝安区西乡街道南昌社区航城大道
华丰国际机器人产业园B栋六层601-605